Tema 7

Optimización y documentación

Entornos de Desarrollo

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Refactorización 5
2.1 Concepto. Limitaciones 5
2.2 Patrones de refactorización más usuales 6
2.3 Refactorización y pruebas 7
2.4 Herramientas de ayuda a la refactorización 7
3. Control de versiones 10
3.1 Desarrollos colectivos 10
3.2 Herramientas de control de versiones. Utilidad. Características. Estructura (cliente/servidor). Repositorio 11
3.3 Clientes de control de versiones. Descarga de ficheros inicial. Modificación de ficheros. Actualización de ficheros en local. Actualización de ficheros en el repositorio .. 11
3.4 Diferencias entre versiones. Restauración de versiones anteriores 16
3.5 Resolución de conflictos. Historial de versiones 16
4. Documentación 18
4.1 Uso de comentarios 18
4.2 Herramientas integradas en el entorno de desarrollo para generar documentación automática de clases 20
4.3 Alternativas 21
5. Resumen 22
Glosario 23
Bibliografía 24

Objetivos de la unidad

- Aprender en qué consiste el concepto de refactorización, así como las técnicas fundamentales y cómo los entornos de desarrollo facilitan su realización.
- Descubrir el concepto de sistema de control de versiones, tomando contacto con los comandos y funciones básicos.
- Conocer las técnicas de documentación de código fuente, así como las razones por las que es tan importante que los programadores realicen esta tarea.

1. Introducción

El **código fuente** es el producto resultante del trabajo de desarrollo. Suele ser construido por varias personas en un periodo largo de tiempo. Durante el proceso, las primeras versiones de código van modificándose para incorporar mejoras o corregir errores, complicándose su diseño y estructura. La refactorización es la técnica que se utiliza para mejorar la estructura del código.

Otra de las dificultades existentes en el desarrollo de *software* tiene relación con el trabajo en grupo y el control de cambios. Los sistemas de control de versiones son las herramientas idóneas para resolver este tipo de problemas.

El código fuente puede ser difícil de comprender. La documentación en general y los comentarios de código en particular ayudan a entenderlo. Todo buen programador debe ser capaz de escribir unos comentarios precisos que ayuden a la comprensión del código.

Tech Hub <FP> euroformac

2. Refactorización

Cualquiera que haya programado habrá observado que los programas se parecen bastante a las mesas de trabajo. Al principio están ordenadas y despejadas, pero a medida que pasa el tiempo el desorden y el caos se apoderan de ellas.

Los programas, aunque hayan sido escritos con cuidado, tienden a volverse desordenados a medida que se van modificando para corregir errores o incluir nueva funcionalidad. Al código fuente que, tras sufrir modificaciones y correcciones sucesivas, se ha deteriorado estructuralmente (pese a que funciona correctamente) se le denomina popularmente **código spagueti**. Para detener y revertir esta circunstancia del código se puede utilizar la técnica de programación conocida como **refactorización.**

2.1 Concepto. Limitaciones

La **refactorización** es un conjunto de técnicas que consisten en modificar un programa que está funcionando correctamente para eliminar complicaciones innecesarias, mejorar su legibilidad, hacerlo eficiente y, en resumen, facilitar su comprensión y mantenimiento.

Una **refactorización de código** no implica dotarlo de más funcionalidad ni eliminar ningún error funcional. Una refactorización de código mejora su estructura y legibilidad.

Un código refactorizado está ordenado y "limpio"

2.2 Patrones de refactorización más usuales

La refactorización se compone de varias técnicas. No todas necesitan ser aplicadas siempre, quedando a la decisión del programador decidir cuáles utilizar. Algunas de las técnicas más utilizadas son las siguientes:

- **Correcta composición de métodos**. Creación de métodos o funciones para encapsular funcionalidad y eliminación de código duplicado. Eliminación de métodos demasiado cortos si no son necesarios. Utilización de variables locales para almacenar los retornos en lugar de los parámetros.
- **Correcta distribución de funcionalidad entre clases**. Conseguir que cada clase tenga la funcionalidad que le corresponde únicamente.
- **Organización de los datos.** Utilizar objetos para representar datos. Encapsulación de datos. Uso de constantes.
- **Simplificación de las expresiones condicionales.** Descomposición de condiciones. Eliminación de duplicidades. Uso de aserciones. Control de objetos nulos.
- **Simplificación en la llamada a métodos.** Asignación correcta de nombres de métodos. Eficiencia en el uso de parámetros. Uso de excepciones en lugar de códigos de error.
- **Optimización de la herencia**. Ubicación correcta de atributos y métodos en la jerarquía. Correcto uso de herencia y de delegación. Uso de interfaces.
- **Denominación correcta.** Asignación de nombres descriptivos y precisos a todos los elementos de los programas: variables, constantes, métodos, objetos, clases, etc.

La refactorización implica tener un buen conocimiento de la programación en sus diversos paradigmas. Al igual que ocurre en otras disciplinas, hacer las cosas simples es difícil. Refactorizar es una tarea de programación avanzada, pero no es necesario tratar de realizar toda la refactorización posible de golpe. Se puede comenzar por pequeños cambios que ya implican una mejor en el código. Cada programador tiene su nivel y debe acomodarse a él e intentar mejorarlo poco a poco.

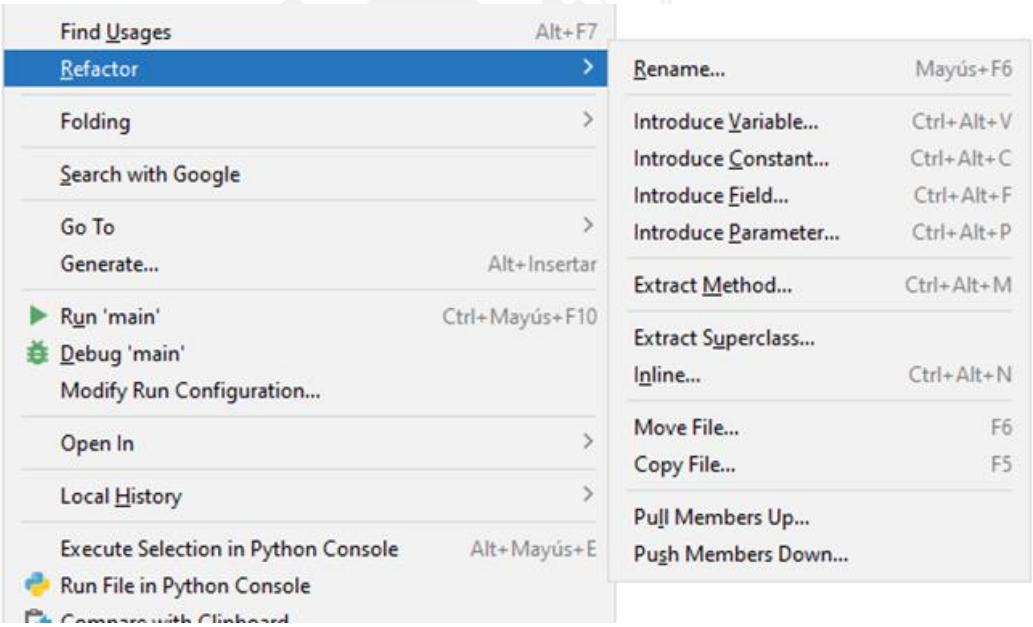
2.3 Refactorización y pruebas

Una **refactorización de código no implica cambios en su funcionamiento**, pero si está mal hecha puede generar errores. Para evitarlo hay que considerar las siguientes acciones:

- Se debe poder deshacer los cambios realizados como consecuencia de la refactorización. Los sistemas de control de versiones proporcionan esta funcionalidad.
- Se debe probar el código modificado. Una vez refactorizado el código es conveniente realizar las pruebas que sean necesarias para verificar que sigue funcionando correctamente (pruebas unitarias, de regresión o de humo).

2.4 Herramientas de ayuda a la refactorización

Los **entornos de desarrollo** proporcionan herramientas para realizar la refactorización de código de manera asistida. La programación en Python,en *PyCharm*, no es una excepción. A continuación, se muestran algunas de las herramientas disponibles en este IDE. Son visibles cuando se selecciona código y se pulsa el botón derecho.



Acciones de refactorización proporcionadas por PyCharm

Como ejemplo, se muestra la refactorización del siguiente bloque de código:

```
importe_minimo = 0

importes = [10000, 20000, 3000, 15000]

mayor = importe_minimo
for importe in importes:
    if importe>mayor:
        mayor=importe

print("El importe mayor es",mayor)
```

La primera refactorización consiste en renombrar la variable importe_minimo para que aparezca como una constante. Pulsando el botón derecho sobre la variable y seleccionando la opción "Refactor/Rename…"se puede renombrar la variable, afectando el cambio a todas las referencias a la misma.

```
IMPORTE_MINIMO = 0

importes = [10000, 20000, 3000, 15000]

mayor = IMPORTE_MINIMO
for importe in importes:
    if importe>mayor:
        mayor=importe

print("El importe mayor es",mayor)
```

La siguiente refactorización consiste en extraer un método de la parte principal del algoritmo. Para ello, se deben seleccionar las líneas que se desean incluir en el método, pulsar el botón derecho del ratón, seleccionar la opción "***Refactor/Extract Method…***" y asignar un nombre al método. El resultado obtenido es el siguiente:

```
IMPORTE_MINIMO = 0

importes = [10000, 20000, 3000, 15000]

def calcular_mayor():
    global mayor
    mayor = IMPORTE_MINIMO
    for importe in importes:
        if importe > mayor:
            mayor = importe

calcular_mayor()
```

```
print("El importe mayor es",mayor)
```

Como se puede observar, se ha generado un método e inmediatamente después su invocación. Posiblemente la solución generada no sea la mejor posible, pero el código sigue funcionando correctamente y se ha agrupado la funcionalidad en una función, tal y como se deseaba.

3. Control de versiones

Los **sistemas de control de versiones** son herramientas fundamentales dentro del desarrollo y de la ingeniería de software. Permiten mantener control sobre el código fuente y el resto de los ficheros del sistema, tanto desde el punto de vista de su evolución como del relacionado con el trabajo en grupo.

Son varios los sistemas de control de versiones existentes, pero todos comparten su misión fundamental, que es facilitar el control de los cambios y permitir el trabajo en grupo.

Gracias a los sistemas de control de versiones se puede:

- Disponer de un lugar común en el compartir el código (repositorio).
- Disponer de copias ordenadas del código fuente para poder restaurarlas en caso de necesidad.
- Tener información histórica sobre los cambios que se han ido realizado en el código a lo largo del ciclo de vida del proyecto.
- Trabajar en grupo, minimizando el impacto de flujo de trabajo de unos programadores en otros.

3.1 Desarrollos colectivos

El desarrollo de aplicaciones complejas implica la participación de muchas personas. Esto supone un hándicap, ya que todo el código programado tiene que compartirse y ponerse en común, con los problemas logísticos que esto puede suponer.

Los **sistemas de control de versiones** facilitan este trabajo. Gracias al uso de repositorios y de ramas, cada usuario (programador) puede efectuar cambios en partes del programa y que estos no afecten al resto hasta que dichos cambios estén terminados y probados.

Más info

Para profundizar en el conocimiento del concepto de rama en relación con el control de versiones, se puede leer el siguiente artículo: Ver enlace.

3.2 Herramientas de control de versiones. Utilidad. Características. Estructura (cliente/servidor). Repositorio

Hay muchos tipos de sistemas de control de versiones. A lo largo de la breve historia de la programación han ido apareciendo nuevos sistemas que han ganado popularidad a la vez que otros caían en el olvido.

Principalmente, los sistemas de control de versiones se dividen en:

- **Centralizados**. Las copias del proyecto se almacenan en un servidor central.
- **Distribuidos**. Las copias del proyecto se almacenan en un servidor y en cada uno de los integrantes del equipo de trabajo.

Muchos de los sistemas de control de versiones más utilizados en la actualidad son distribuidos, como **Git** o **Mercurial**, pero también hay sistemas centralizados o basados en una arquitectura cliente-servidor, como **CVS** o **Apache Subversion.**

Como elemento común, todos estos sistemas se basan en el concepto de **repositorio**, que es el lugar en el que se almacena el código del proyecto junto con todos los cambios que se han producido en el mismo.

3.3 Clientes de control de versiones. Descarga de ficheros inicial. Modificación de ficheros. Actualización de ficheros en local. Actualización de ficheros en el repositorio

El programador es el usuario principal de los sistemas de control de versiones. Para interactuar con el sistema necesita una herramienta, conocida como **cliente**. Como es habitual, existen muchas alterativas, tantas al menos como sistemas existentes.

El sistema de control de versiones distribuido más utilizado es **Git**. Fue diseñado por *Linus Torvalds*, el creador de *Linux*, y se trata de un sistema de *software* libre.

Logotipo de git

Git se puede utilizar desde línea de comandos, a través de una interfaz de usuario oficial conocido como **Git Gui** o por medio de clientes de terceros como **GitHub Desktop, Sourcetree** o **GitKraken**.

El primer paso para trabajar con un proyecto consiste en instalar el *software* cliente del sistema de control de versiones. Por ejemplo, Git se puede descargar e instalar desde el siguiente enlace: Ver enlace.

Una vez instalado, se puede elegir entre crear un nuevo proyecto (repositorio) o clonar uno existente. Si se trabaja desde línea de comandos, los correspondientes a estas dos acciones son:

- **git init.** Para iniciar un nuevo repositorio.
- **git clone** *path*. Para clonar un repositorio existente.

Por ejemplo, ejecutando el comando **git clone https://github.com/id-Software/DOOM.git** en una consola, se clonará el proyecto original del famoso y genial videojuego DOOM de la cuenta de la empresa que lo creó (id-*Software*) del sitio web de almacenamiento de proyectos Git **github.com**. Una vez clonado, se dispondrá de una copia del código fuente del videojuego en la carpeta desde la que se ejecutó el comando.

Sabías que

github.com es un sitio web que permite crear y administrar repositorios basados en Git de forma gratuita. Es propiedad de Microsoft y resulta muy interesante disponer de una cuenta en dicha plataforma.

Al no tratarse de un proyecto propio y no disponer de los permisos correspondientes, se podrá modificar la copia, pero no el original almacenado en el servidor.

No obstante, suponiendo que se tratase de un proyecto propio, se podría modificar el código para agregarle nueva funcionalidad y/o resolver errores. Cuando esto ocurra, el programador debe hacer dos cosas:

- Hacer firmes los cambios, mediante el comando **commit.**
- Subir los cambios al servidor, mediante el comando **push.**

La acción de confirmación **commit** hace firmes los cambios en la base de datos local del repositorio. Incluso una vez realizado, el repositorio principal no ha sido modificado. No es hasta que se realiza la función **push** cuando los cambios se hacen efectivos en el servidor.

En el siguiente ejemplo se documentan los pasos a seguir para crear un repositorio, agregarle nuevo contenido y publicado en una cuenta de github.com.

Antes de comenzar:

- Se debe disponer de una cuenta en github.com
- Se debe haber instalado Git en el ordenador.

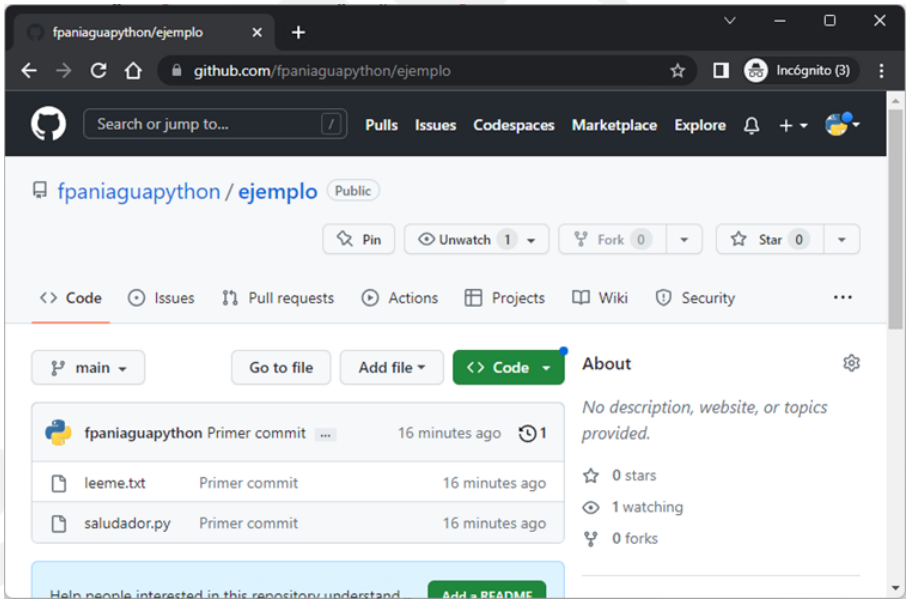
Pasos:

- Ejecutar **git init** desde la carpeta del nuevo proyecto. La carpeta en ese momento se encuentra vacía.
- Crear los ficheros **saludador.py** y **leeme.txt** en la carpeta. El contenido no es importante, pero se recomienda que el fichero .py contenga código Python y el .txt información general.
- Ejecutar **git add *.***. Se agregarán los ficheros a una lista de cambios pendientes.
- Ejecutar **git commit -m "Primer commit"**. Los cambios se harán firmes en el repositorio local. La salida producida se muestra a continuación. Se pueden observar los nombres de los ficheros agregados al repositorio.

```
G:\ejemplos>git commit -m "Primer commit"
[master (root-commit) b5e1f9d] Primer commit
 2 files changed, 3 insertions(+)
 create mode 100644 leeme.txt
 create mode 100644 saludador.py
```

- Crear un nuevo repositorio en github.com con el nombre **ejemplo**.
- Ejecutar **git remote add origin https://github.com/*fpaniaguapython*/ejemplo.git**, siendo *fpaniaguapython* el identificador de la cuenta de github.com utilizado para el ejemplo (recuerda que el identificador que aparecerá será el usuario que crees nuevo o con el que accedes a en este repositorio). Este comando vincula el repositorio local con el remoto.
- Ejecutar **git branch -M main**. Este comando crea una rama (*branch*) principal.
- Ejecutar **git push -u origin main**. Este comando copia los ficheros locales en el repositorio del servidor.

Una vez realizado el proceso completo, en el repositorio almacenado en github.com se podrán observar los ficheros.



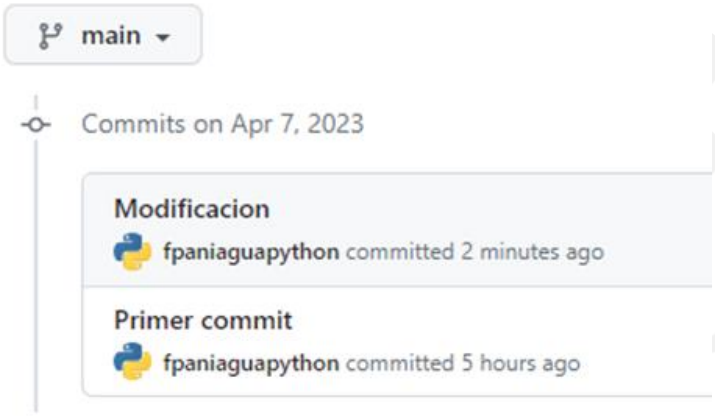
Ficheros almacenados en el servidor remoto de Git

Una vez creado un repositorio y alojado en un servidor, la gestión de cambios se puede realizar de la siguiente manera:

- Introducir cambios en los ficheros del proyecto. En el ejemplo se modifica **saludador.py.**

- Ejecutar **git commit -a -m "Modificacion"** para hacer firmes los cambios.
- Ejecutar **git push -u origin main** para subir los cambios al servidor.

En el servidor se puede comprobar como el repositorio ahora contiene dos ***commits.***



Commits realizados en el repositorio

Más info

Se puede aprender a crear repositorios en el servidor en el siguiente manual: Ver enlace.

El trabajo con el sistema de control de versiones se puede realizar desde una terminal introduciendo los comandos de manera manual, como se ha podido observar. Esta forma de trabajo es elegida por muchos programadores. No obstante, el uso de clientes con interfaz gráfica de usuario puede resultar más sencilla, aunque para los usuarios más avanzados pueda parecer limitada. Por otra parte, los IDEs suelen integrar el uso de sistemas de control de versiones como parte de sus herramientas.

3.4 Diferencias entre versiones. Restauración de versiones anteriores

Como ya se ha comentado, los sistemas de control de versiones almacenan cada uno de los cambios que se han efectuado en los repositorios.

En git, ejecutando **git log** se muestran los *commits* realizados. Siguiendo con el ejemplo anterior, el sistema muestra que se han realizado dos *commits:* "Primer commit" y "Modificacion", cada uno de ellos identificado por un identificador único, marcado en el siguiente código en negrita.

```
commit 9e46531f96fdc4fdd49b6f3f57c84fd92b61d6ac (HEAD -> main,
origin/main)
Author: Fernando Paniagua <fernando.paniagua.python@gmail.com>
Date:   Fri Apr 7 19:11:59 2023 +0200

    Modificacion

commit b5e1f9d643106b5eb8086692bad009b971e7f41b
Author: Fernando Paniagua <fernando.paniagua.python@gmail.com>
Date:   Fri Apr 7 13:19:46 2023 +0200

    Primer commit
```

Si se desea restaurar el repositorio local en el estado en el que estaba tras el primer *commit* se debe ejecutar el siguiente comando (se debe prestar atención al identificador del *commit*):

```
git reset --hard b5e1f9d643106b5eb8086692bad009b971e7f41b
```

Los ficheros del repositorio se actualizarán al estado en el que se encontraban tras ese *commit* inicial.

3.5 Resolución de conflictos. Historial de versiones

Es fácil que aparezcan conflictos a la hora de utilizar un sistema de control de versiones, ya que pueden ser muchos los usuarios que trabajen en el mismo proyecto y se puede dar la circunstancia de que estén modificando el mismo fichero. En git, el comando **merge** es el adecuado para resolver este tipo de problemas.

Hay que recordar que los sistemas de control de versiones mantienen almacenados todos los estados por los que ha pasado el proyecto, siempre y cuando estos se hayan subido al servidor. De esta manera, se dispone de un

historial de versiones (tanto de desarrollo como finales) del que se puede obtener todos los ficheros fuentes.

Más info

Se puede profundizar más en la resolución de conflictos en Git en el siguiente artículo: Ver enlace.

4. Documentación

Una de las actividades que forman parte del desarrollo de software es la **documentación**. La documentación hace referencia a toda información que forma parte del proyecto, recogida en cualquier tipo de soporte y que no incluye el código fuente, aunque este también incluya una suerte de documentación conocida como **comentarios**.

Muchos programadores, especialmente los más inexpertos, consideran que documentar el código es una actividad ajena a la programación y opcional. No es así. La documentación forma parte de las buenas prácticas de la programación.

4.1 Uso de comentarios

Se conoce como **comentarios** al texto escrito en lenguaje natural incluido en el código fuente que no tiene ninguna utilidad a nivel de ejecución (no tiene que cumplir la sintaxis del lenguaje) y que pretende explicar dicho código para facilitar su comprensión, uso y mantenimiento.

Hay dos **tipos de comentarios que se incluyen en el código**: los que explican el código a nivel interno y los que dan indicaciones sobre cómo se debe utilizar el código por parte de otros programas.

En Python los comentarios se indican utilizando el carácter **#** (almohadilla). El texto escrito a continuación de una almohadilla en la misma línea no es corregido ni ejecutado, por lo que puede ser escrito en lenguaje natural.

```
#Información de la serie
titulo = 'The stand'
inicio = 2020 #Año de emisión del primer episodio
fin = 2021 #Año de emisión del último episodio
```

Python, además, admite crear bloques de comentarios formados por varias líneas. Para marcar dicho bloque se deberá delimitar por **tres comillas antes y después del contenido**. Las comillas pueden ser **simples o dobles** (conocidas formalmente como altas o inglesas).

```
'''
Este bloque de texto
no es tratado por Python,
considerándose un segmento
de comentarios escrito en
varias líneas.
'''
```

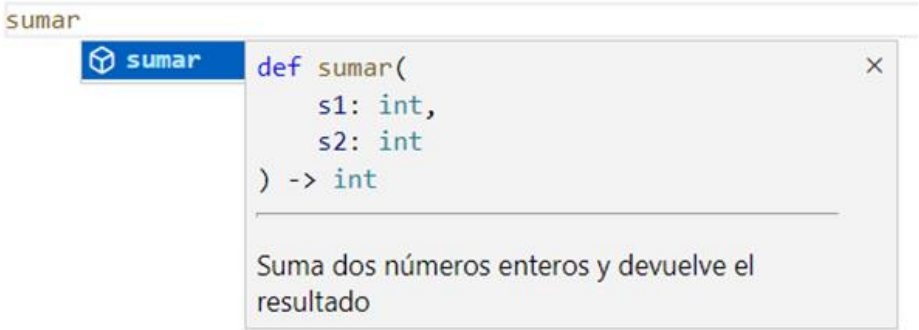
Se puede observar que los comentarios anteriores no son formales en absoluto. Simplemente se indica, mediante la notación adecuada, que determinada información no debe ser tratada como parte del programa.

En Python existe otro tipo de comentario de código más formal conocido como ***docstring***. Mediante el uso de *docstring*, se va a poder documentar las funciones, clases y métodos con el objetivo de facilitar su uso al resto de programadores. Los comentarios creados con *docstring* se muestran cuando se utiliza la función integrada de Python ***help*** o por parte de los editores o IDEs como ayuda contextual.

Para ilustrar este comportamiento, se crea el fichero **calculadora.py** que contiene el siguiente código. Se marca en negrita el comentario escrito con *docstring*. Se diferencia de un comentario convencional en la posición, justo después del nombre de la función.

```
def sumar(s1 : int, s2: int) -> int:
    """
    Suma dos números enteros y devuelve el resultado
    """
    resultado = s1 + s2
    return resultado
```

Al escribir el nombre de la función en Visual Studio Code, se muestra lo siguiente:



Efecto de docstring en Visual Studio Code

Igualmente, desde el entorno interactivo de Python, una vez importado el módulo y la función sumar, usando la función **help** se obtiene el siguiente resultado:

```
>>> help(sumar)
Help on function sumar in module calculadora:

sumar(s1: int, s2: int) -> int
    Suma dos números enteros y devuelve el resultado
```

Como se ha comentado anteriormente, la notación *docstring* se puede utilizar para documentar clases y métodos.

Más info

Se puede aprender más sobre el uso de *docstring* en el siguiente enlace: Ver enlace.

Sabías que

Java dispone de un mecanismo de documentación de clases y métodos oficial conocido como **Javadoc**. Los programadores de Java deben conocer su uso en profundidad.

4.2 Herramientas integradas en el entorno de desarrollo para generar documentación automática de clases

Los IDEs suelen incorporar herramientas para la creación de la documentación del código, automatizando todo lo posible la escritura.

En *PyCharm* pulsando la combinación de teclas CTRL+ENTER sobre el nombre de una función, clase o método, se presenta la opción "*Insert a documentation string stub*" que inserta un esqueleto de documentación utilizando la información disponible.

Por ejemplo, para el código fuente siguiente:

```
def sumar(s1 : int, s2: int) -> int:
    resultado = s1 + s2
    return resultado
```

El generador de documentación genera el siguiente bloque de comentario:

```
def sumar(s1 : int, s2: int) -> int:
    """

    :param s1:
    :param s2:
    :return:
    """
    resultado = s1 + s2
    return resultado
```

Debiendo el programador rellenar los huecos correspondientes a la explicación de la función, de los parámetros y del retorno.

4.3 Alternativas

Cada lenguaje dispone de su propio **sistema de comentarios**, incluyendo la posibilidad de generar documentación a partir de dichos comentarios. Cuando un programador se enfrenta al aprendizaje de un lenguaje nuevo debe conocer y utilizar el sistema de comentarios que le es propio.

Sea como sea, los comentarios son útiles y, casi siempre, necesarios. No solo para facilitar la lectura y comprensión del código por parte del resto de programadores, sino también por parte de su autor, ya que es fácil olvidar por qué se tomó determinada decisión pasados unos días después de hacerlo.

5. Resumen

El **código fuente** de un programa, además de hacer aquello para lo que se construye, debe estar limpio, ser comprensible y estar almacenado con control.

Las **buenas prácticas de programación**, utilizadas junto con las técnicas de refactorización, permiten obtener un código bien estructurado y legible.

Los **comentarios** en el código ayudan a comprender y usar el código. Es tarea de los programadores escribirlo y conseguir que sean útiles.

Por último, el **almacenamiento del código** para tener control sobre sus diferentes cambios y versiones se puede realizar utilizando los sistemas de control de versiones, un tipo de software específicamente diseñado para permitir el trabajo en grupo y controlar los cambios en los programas mientras estos se escriben.

Tech Hub <FP> euroformac

Glosario

- **Código limpio (*clean code*)** Es un término que hace referencia al objetivo de construir código fácil de leer, comprender y mantener. Existen libros dedicados en exclusiva a esta especie de "filosofía" de la programación
- **Control de cambios.** En lo referente al *software*, es el conjunto de técnicas que se llevan a cabo para tener control sobre los cambios que se producen en el código a lo largo del proceso de construcción
- **Clonar repositorio.** Clonar un repositorio consiste en realizar una copia de este. En principio esta copia es independiente del repositorio original, por lo que los cambios que se realicen en el repositorio clonado no tendrán impacto en el de origen.
- **Rama (*branch*).** Es una copia del proyecto que temporalmente está independizada de la rama principal. Permite a un desarrollador trabajar sobre el proyecto sin que los cambios afecten al resto.
- **Repositorio.** Es el lugar en el que se almacena el código fuente de un proyecto junto con el histórico de cambios que ha experimentado durante su creación

Bibliografía

Atlassian. (n.d.). Collaboration software for software, IT and business teams. https://www.atlassian.com/

DataCamp. (n.d.). Learn R, Python & Data Science Online. https://www.datacamp.com/

Git. (n.d.). https://git-scm.com/

GitHub: Let's build from here. (n.d.). GitHub. https://github.com/

Refactoring.Guru. (n.d.). Refactoring and Design Patterns. https://refactoring.guru/

Tech Hub <FP> euroformac